



Technische
Universität
Braunschweig

Decision
Support
Institut für Wirtschaftsinformatik



Script Gadgets

Bypassing XSS Mitigations via Browser Code Reuse

Web Security Group, May 31, 2026

Outline



Part I

Introduction

Motivation

- **Cross-Site Scripting (XSS)** has ranked among the most common web vulnerabilities for years (Lekies et al., 2017).
- Despite research and training, incidence remains high – many apps rely on **mitigations** as a second line of defense.
- These mechanisms mostly target `<script>` tags and event handlers.
- **Paper's thesis:** That is no longer enough for modern JavaScript applications.

Four Classes of XSS Mitigations

Per Lekies et al. (2017) (Section 2.3):

1. **HTML sanitizers** (DOMPurify, Google Closure)
2. **Browser XSS filters** (Chrome, Edge, NoScript)
3. **Web Application Firewalls** (ModSecurity CRS)
4. **Content Security Policy (CSP)**

Three underlying strategies: request filtering, response sanitization, code filtering.

Part II

Script Gadgets

Definition: Script Gadget

Script Gadget (Lekies et al., 2017) (Section 3.3)

A fragment of **legitimate** JavaScript on the site, triggered by **benign HTML** in the document, that processes attacker data in an unsafe way.

- Not injected by the attacker – already present in app or library code.
- Introduces **code-reuse attacks** on the web (analogy: return-to-libc).

Benign HTML

Mitigations block directly executable code but often allow (Lekies et al., 2017) (Section 3.1):

- No `<script>`, no inline handlers
- No `javascript:/data:` URLs in `src/href`
- No executable tags such as `<link rel=import>`, `<meta>`, `<style>`

Listing 1 – benign for filters

```
<div class="greeting"><b>Hello</b> world!</div>
```


DOM Selectors

Benign markup alone does not execute code (Section 3.2):

- Legitimate code reads the DOM (`getElementById`, `querySelector`, `jQuery`).
- Paper example: elements with **tooltip** attributes are decorated.
- Injected markup **matches** selectors → gadget path activates.

Attack Outline

Four steps (Lekies et al., 2017) (Section 3.4):

1. **Injection** – benign HTML (stored/reflected XSS)
2. **Mitigation** – no direct script detected, markup remains
3. **Gadget** – trusted code transforms the payload
4. **Execution** – attacker-controlled JavaScript runs

Gadget Types

Classification (Section 3.5):

- **String manipulation** – e.g. `data-*` → framework attributes (AngularJS, Polymer)
- **Element construction** – `new <script> (jQuery $.globalEval)`
- **Function creation** – `new Function(...)`; often combined with `unsafe-eval` CSP

Complex attacks **chain** multiple gadgets.

Part III

Bypassing Mitigations

Bypass Strategy (Overview)

Gadgets use **different HTML tags/attributes** than classic XSS payloads (Lekies et al., 2017):

- Request filtering: only “evil” patterns, not gadget paths
- Response sanitization: whitelist keeps markup; library makes it dangerous
- Code filtering (CSP): execution via **trusted** code

Bypass: Request Filtering

WAFs, NoScript (Section 4.2):

- Detect `<script>`, `onerror`, `eval`, `document.cookie`, ...
- Gadgets use **allowed** attributes (`data-html`, `data-bind`, ...)
- Frameworks split exploits (Knockout example, Listing 13): harmless parts together = attack

Bypass: Response Sanitization

HTML sanitizers, IE/Edge XSS filter (Section 4.3):

- Sanitizer: only whitelisted attributes (`class`, `id`) – gadget abuses these
- jQuery Mobile + DOMPurify (Listing 14): `data-role=popup` injects comment → escape → XSS
- Gadgets are “safe” by definition until library logic activates them

Bypass: Code Filtering (CSP)

CSP and XSS Auditor (Section 4.4):

- `unsafe-eval`: gadgets using `eval`/templates (e.g. Underscore)
- `strict-dynamic`: trusted libs create new `<script>` elements (jQuery)
- Nonce/whitelist CSP: expression parsers in AngularJS, Aurelia, Polymer
- **13/16** frameworks studied: bypass of `strict-dynamic`

Empirical Results

- 16 JS libraries/frameworks analyzed manually (jQuery, AngularJS, Vue, Bootstrap, ...)
- Gadgets in **almost all**; React had no usable gadgets
- Automated study: Alexa Top 5000, ~650 000 URLs
- **19.88%** of domains: verified gadgets (conservative lower bound)
- 4 352 491 sink executions with DOM-sourced data measured

Lekies et al. (2017)

Part IV

Case Study

Example: Bootstrap Tooltip

Supplementary course example Novaes (2026) – illustrates the paper:

- **Stored XSS:** unsanitized HTML in profile/tooltip title
- **Gadget:** Bootstrap renders tooltip HTML (trusted library)
- **Bypass:** `sanitize: false` disables internal allowlist protection
- Trigger on hover; exfiltration e.g. via `<img onerror> + fetch`

Maps to Paper Sections 3–4; use the course lab app for a live demo if required.

Part V

Conclusion

Countermeasures

- Fix the flaw: context-aware encoding/sanitization **before** the DOM
- Do not disable library defaults (`sanitize: true`)
- **CSP** helps but is not enough alone: review `strict-dynamic` and `unsafe-eval`
- Defense in depth: sanitizer + CSP + safe API usage
- PoC collection: <https://github.com/google/security-research-pocs>

Takeaways

Key message (Lekies et al., 2017)

The assumption “injected code $\neq$ benign markup” is false in modern web apps.

- Script gadgets bypass all four mitigation classes.
- Trusted frameworks can be abused for XSS execution.
- Secure development is still required – mitigations do not replace it.

Questions?

Thank you for your attention!

Lekies et al., CCS 2017 Lekies et al. (2017)



References I

- Lekies, S., Kotowicz, K., Groß, S., Vela Nava, E. A., and Johns, M. (2017). Code-reuse attacks for the web: Breaking cross-site scripting mitigations via script gadgets. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 1709–1722.
- Novaes, L. (2026). Script gadget demo: Bootstrap tooltip stored xss. Course documentation (pdf_from_friends.pdf). Lab demo: Bootstrap tooltip, ports 8000/9000.